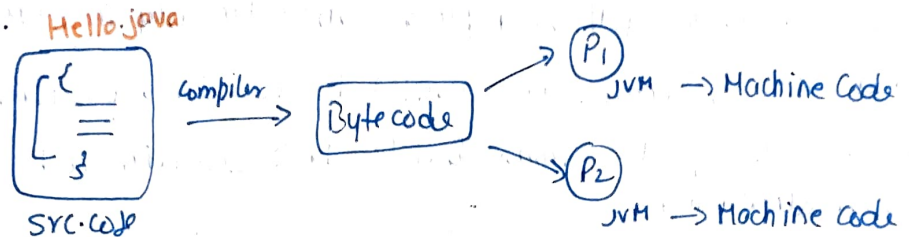
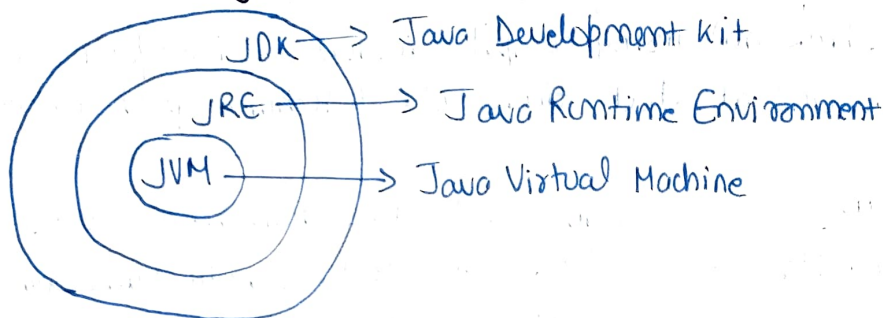


Lecture - 2 JVM vs JDK vs JRE

Byte code is platform independent. JVM is responsible to make machine code from Byte code.



* Java Hierarchy



JVM (Java Virtual Machine) is a program that runs your Java code. When you write java, it gets converted into a special format called byte code, & the JVM reads that byte code and turns it into instructions your computer understands. Because of the JVM, the same java program can run on different computers without changing code.

→ There are 2 types of softwares to convert code to Machine code:-

- 1.) Compiler
- 2.) Interpreter

→ Compiler reads whole source code and converts it to machine code and then runs it on system.

→ Interpreter converts code line by line → so it will convert one line to machine code & execute it then it will convert line 2 & so on.

→ C/C++ is compiled language

→ Python is interpreted language

But java is neither purely compiled nor interpreted language rather Java is both compiled and interpreted language.

JVM

Byte code $\rightarrow$ Interpreter $\rightarrow$ Machine Code $\rightarrow$ This used to be the case in the 1990s

when Java started getting popular and why only in JVM we used interpreter not compiler, because we wanted to run Java program as soon as possible and since Interpreter immediately starts executing code as java already uses a slow process.

Java $\rightarrow$ Compiler $\rightarrow$ Bytecode $\rightarrow$ P1

Bytecode $\rightarrow$ JVM $\rightarrow$ Machine code $\rightarrow$ P2

So to make Java platform independent we do 2 process to make it work that already takes time and using compiler instead of interpreter would have taken more time and at that time lots of technical problems were there.

* Earlier problems

- 1.) Slow Hardware
- 2.) Disk slow
- 3.) RAM Limited

and due to these limitations we used interpreter in JVM to quickly run Java code but with time problems got eradicated due to which we combined a compiler with Interpreter in JVM.

JVM $\rightarrow$ Interpreter + JIT (Just in Time) compiler

$\rightarrow$ both work in accordance with each other to convert bytecode to Machine code.

$\rightarrow$ JVM uses hybrid model of Interpreter + Compiler

$\rightarrow$ So JVM analyses which part of code will be frequently used. That part will be compiled by JIT into Machine code and the code that is not frequently used will be interpreted by interpreter.

$\rightarrow$ C++/C is slightly faster than Java as C/C++ code immediately gets converted to Machine code but Java is slightly slower as it first compiles Java code to bytecode & then to Machine code. But once Java code runs it provides such as :-

- 1.) Portability
- 2.) Simple
- 3.) Security

and these days performance diff b/w c/c++ and Java is very negligible.

* Roles of JVM

- 1.) Convert byte code to Machine Code (Interpreter + JIT compiler)
- 2.) Security (Sandbox Model) → so code does not execute on our system but inside secured environment of JVM to monitor unauthorized access.
- 3.) Garbage Collection → this process occurs inside JVM.

* JRE

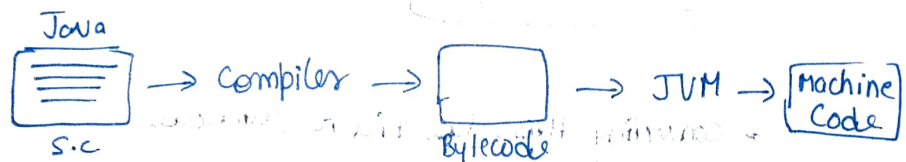
↳ JVM + class libraries [these are pre built libraries that comes pre-installed with JDK.]

→ so bytecode only contains import statements but when actually JVM runs bytecode then actual code from those paths are included.

* JDK → it includes everything to write & run Java code.

JDK → JRE + compiler + Debugger + many more (Javac)

now using JDK we will be able to do



* JSE, JEE, JME → these are simply different standards of Java.

JSE → Java Standard Edition

JEE → Java Enterprise Edition

JME → Java Micro Edition

* JSE → Java SE is the core Java. It is used to develop basic Java applications. It includes Core Libraries, JVM, JRE, Basic tools. It is used for desktop applications, console applications, Backend logic, Basic Java programs

eg → Calculator app, basic java program.

* **Java EE (JEE)** → Java Enterprise Edition now called Jakarta EE. It is used to build web applications and enterprise applications. It includes everything from Java SE + extra libraries like:

- 1.) Servlet
- 2.) JSP
- 3.) Spring
- 4.) Hibernate

and it is used for web applications, banking systems, REST APIs, e-commerce websites etc.

- 5.) Web Services
- 6.) EJB
- 7.) JPA

eg → Amazon website backend, spring boot web app. etc.

* **JME** → Java Micro Edition is used for small devices with low memory & low power. It is used for

- Mobile phones (old keypad phones)
- Embedded Systems
- IoT devices etc

Compiling Java File to Class File

eg → Hello.java → compiling
→ `javac Hello.java` → Hello.class

→ converting this class file to runcode

`java Hello`